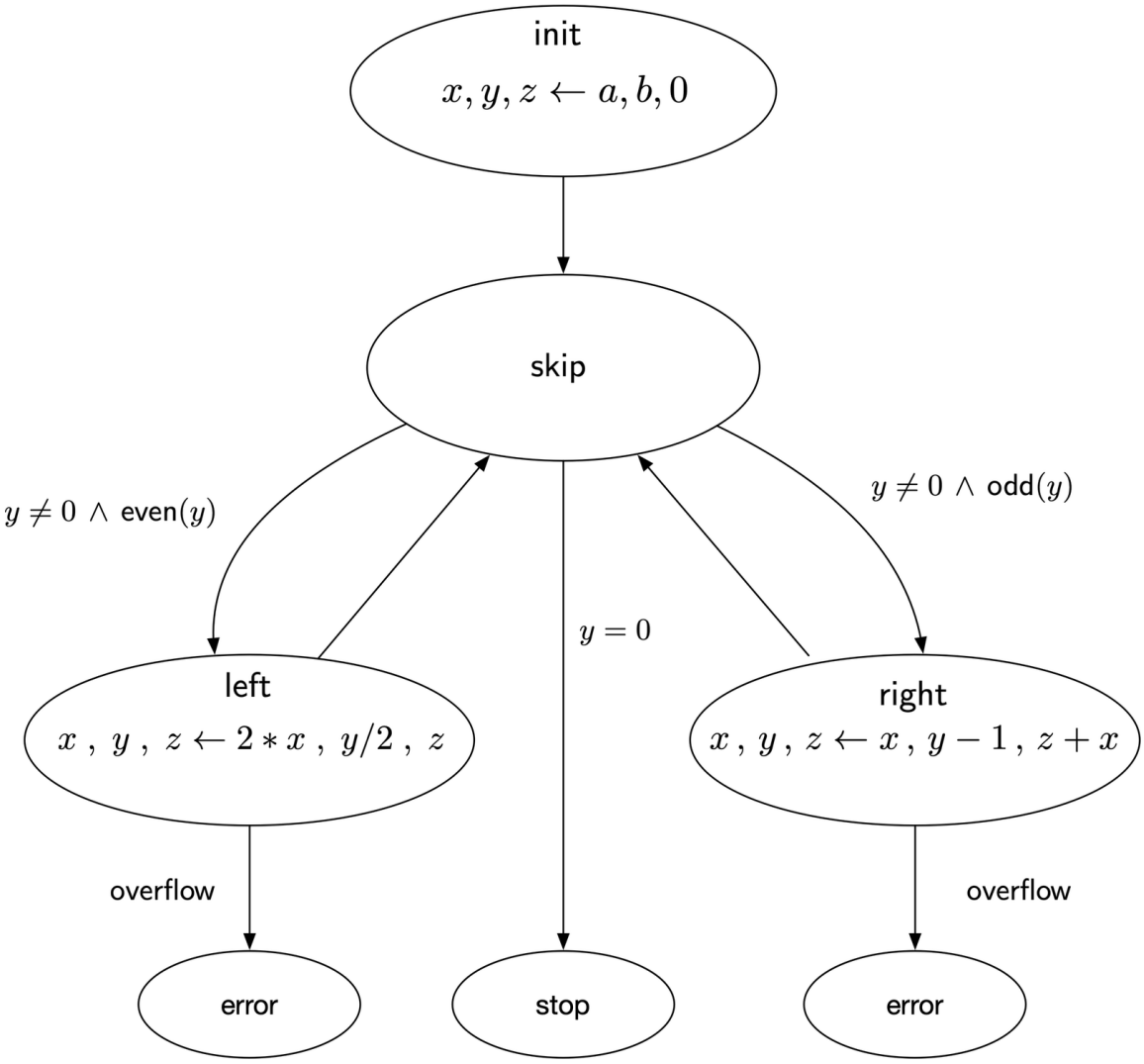
In [ ]:
```
import marimo as mo
from pysmt.shortcuts import Symbol, BV, And, Equals, get_model, Int, GE,
Or, GT, BVSGT, BVSLE, BVSLT, BVSGE, Not, BVSRem, BVLShl, BVLShr, BVSub,
BVAdd, Solver, BVURem, BVUGE, BVULT, BVMul, NotEquals
from pysmt.typing import BVType, INT
```

Variáveis globais

Exercício 3

Considere o problema descrito no documento +Lógica Computacional: Multiplicação de Inteiros . Nesse documento usa-se um "Control Flow Automaton" como modelo do programa imperativo que calcula a multiplicação de inteiros positivos representados por vetores de bits.



- a. Construir um SFOTS, usando BitVec's de tamanho n , que descreva o comportamento deste autómato; para isso identifique e codifique em `Z3` , as

variáveis do modelo, o estado inicial , a relação de transição e o estado de erro.

- b. Usando k-indução verifique nesse SFOTS se a propriedade $x \times y + z = a \times b$ é um invariante do seu comportamento.
- c. Usando k-indução no SFOTS acima e adicionando ao estado inicial a condição $a < 2^{n/2} \wedge b < 2^{n/2}$, verifique a segurança do programa; nomeadamente prove que, com tal estado inicial, o estado de erro nunca é acessível.
- d. Codifique a propriedade de animação que exprime que o programa termina quaisquer que sejam os valores iniciais de a e b e prove que ela se verifica no SFOTS que construiu.

Estados

```
assert a >= 0 and b >= 0
x , y, z = a , b , 0
0: while not y == 0:
1:   if even(y):
2:       x , y , z = x << 1 , y >> 1 , z
     else:
3:       x , y , z = x , y - 1, z + x
4: stop
5: erro
```

Parte a

Construção do SFOTS

Começamos pela declaração dos estados e a inicialização do primeiro estado, de acordo com as regras indicadas no início do programa.

In [ ]:
```
def declare(i):
    state = {}
    state['pc'] = Symbol('pc'+str(i),INT)
    state['x'] = Symbol('x'+str(i),BVType(N))
    state['y'] = Symbol('y'+str(i),BVType(N))
    state['z'] = Symbol('z'+str(i),BVType(N))
    state['a'] = Symbol('a'+str(i),BVType(N))
    state['b'] = Symbol('b'+str(i),BVType(N))
    return state

def init(state, i=None):
    constraints = [
        Equals(state['pc'], Int(0)),
        Equals(state['x'], state['a']),
        Equals(state['y'], state['b']),
        Equals(state['z'], BV(0, N))
    ]
```

```
    if SIGNAL:
        constraints.append(BVSGE(state['a'], BV(0,N)))
        constraints.append(BVSGE(state['b'], BV(0,N)))

    if i is not None:
        constraints.append(Equals(state['a'], i['a']))
        constraints.append(Equals(state['b'], i['b']))

    return And(constraints)
```

Transições

In [ ]:

```
def equal(curr, prox, list_symbol):
    return [Equals(prox[c], curr[c]) for c in list_symbol]

def error(state):
    return Equals(state['pc'], Int(5))

def stop(state):
    return Equals(state['pc'], Int(4))

def trans(curr,prox):
    return Or(
        # casos de erro
        And(
            Equals(curr['pc'], Int(2)),
            BVSLT(BVLShl(curr['x'], 1), BV(0,N)) if SIGNAL else
BVULT(BVLShl(curr['x'], 1), curr['x']),
            error(prox),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])

        ),
        And(
            Equals(curr['pc'], Int(3)),
            BVSLT(BVAdd(curr['z'], curr['x']), BV(0,N)) if SIGNAL else
BVULT(BVAdd(curr['z'], curr['x']), curr['z']),
            error(prox),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        # loop
        And(
            Equals(curr['pc'], Int(0)),
            Not(Equals(curr['y'], BV(0, N))),
            Equals(prox['pc'], Int(1)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        # caso em que y é par
        And(
            Equals(curr['pc'], Int(1)),
            Equals(BVSRem(curr['y'], BV(2, N)), BV(0, N)) if SIGNAL else
Equals(BVURem(curr['y'], BV(2, N)), BV(0, N)),
            Equals(prox['pc'], Int(2)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
```

```
        And(
            Equals(curr['pc'], Int(2)),
            Equals(prox['pc'], Int(0)),
            Not(BVSLT(BVLShl(curr['x'], 1), BV(0,N))) if SIGNAL else
Not(BVULT(BVLShl(curr['x'], 1), curr['x'])),
            Equals(prox['x'], BVLShl(curr['x'], 1)),
            Equals(prox['y'], BVLShr(curr['y'], 1)),
            *equal(curr, prox, ['z', 'a', 'b'])
        ),
        # caso em que y é ímpar
        And(
            Equals(curr['pc'], Int(1)),
            Equals(BVSRem(curr['y'], BV(2, N)), BV(1, N)) if SIGNAL else
Equals(BVURem(curr['y'], BV(2, N)), BV(1, N)),
            Equals(prox['pc'], Int(3)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        And(
            Equals(curr['pc'], Int(3)),
            Equals(prox['pc'], Int(0)),
            Not(BVSLT(BVAdd(curr['z'], curr['x']), BV(0,N))) if SIGNAL else
Not(BVULT(BVAdd(curr['z'], curr['x']), curr['z'])),
            Equals(prox['y'], BVSub(curr['y'], BV(1, N))),
            Equals(prox['z'], BVAdd(curr['z'], curr['x'])),
            *equal(curr, prox, ['x', 'a', 'b'])
        ),
        # caso de terminação
        And(
            Equals(curr['pc'], Int(0)),
            Equals(curr['y'], BV(0, N)),
            Equals(prox['pc'], Int(4)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        And(
            Equals(curr['pc'], Int(4)),
            Equals(prox['pc'], Int(4)),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        ),
        And(
            error(curr),
            error(prox),
            *equal(curr, prox, ['x', 'y', 'z', 'a', 'b'])
        )
    )
```

Gera traço

In [ ]:

```
def gera_traco(initial, kmax=300):

    with Solver(name="z3") as s:

        # states = [declare(i) for i in range(k)]
        states = [declare(0)]
        s.add_assertion(init(states[0], initial))
```

```
        for c in range(kmax):
            s.push()
            s.add_assertion(Or(Equals(states[-1]['pc'], Int(4)),
Equals(states[-1]['pc'], Int(5))))


            if s.solve():
                model = s.get_model()
                w = len(str(model.get_value(states[-1]
['x'])).constant_value()))
                for i, st in enumerate(states):
                    pc_val = model.get_value(st['pc']).constant_value()
                    x_val = model.get_value(st['x']).constant_value()
                    y_val = model.get_value(st['y']).constant_value()
                    z_val = model.get_value(st['z']).constant_value()
                    print(f"Estado {i:>3} : pc = {pc_val:>1}, x = {x_val:>
{w}}, y = {y_val:>{w}}, z = {z_val:>{w}}")
                s.pop()
                return

            s.pop()
            states.append(declare(c+1))
            s.add_assertion(trans(states[c], states[c+1]))

        print("Não foi encontrado um traço válido")
```

Teste

In [ ]:
```
i = {'pc': Int(0), 'a' : BV(2, N), 'b' : BV(25, N)}
gera_traco(i)
```

```
Estado   0 : pc = 0, x =  2, y = 25, z =  0
Estado   1 : pc = 1, x =  2, y = 25, z =  0
Estado   2 : pc = 3, x =  2, y = 25, z =  0
Estado   3 : pc = 0, x =  2, y = 24, z =  2
Estado   4 : pc = 1, x =  2, y = 24, z =  2
Estado   5 : pc = 2, x =  2, y = 24, z =  2
Estado   6 : pc = 0, x =  4, y = 12, z =  2
Estado   7 : pc = 1, x =  4, y = 12, z =  2
Estado   8 : pc = 2, x =  4, y = 12, z =  2
Estado   9 : pc = 0, x =  8, y =  6, z =  2
Estado  10 : pc = 1, x =  8, y =  6, z =  2
Estado  11 : pc = 2, x =  8, y =  6, z =  2
Estado  12 : pc = 0, x = 16, y =  3, z =  2
Estado  13 : pc = 1, x = 16, y =  3, z =  2
Estado  14 : pc = 3, x = 16, y =  3, z =  2
Estado  15 : pc = 0, x = 16, y =  2, z = 18
Estado  16 : pc = 1, x = 16, y =  2, z = 18
Estado  17 : pc = 2, x = 16, y =  2, z = 18
Estado  18 : pc = 0, x = 32, y =  1, z = 18
Estado  19 : pc = 1, x = 32, y =  1, z = 18
Estado  20 : pc = 3, x = 32, y =  1, z = 18
Estado  21 : pc = 0, x = 32, y =  0, z = 50
Estado  22 : pc = 4, x = 32, y =  0, z = 50
```

Parte b

Indução $x \times y + z = a \times b$

0-Indução

> Para todo o estado x

- (a) $\neg\,P(x)$ é inconsistente

1-Indução

> Para todo par de estados x, y deve-se verificar:

- (a) $\quad\mathsf{init}(x)\,\land\,\neg P(x)\quad$ é inconsistente
- (b) $\quad\mathsf{trans}(x,y) \,\land\, P(x)\,\land\,\neg\,P(y)\quad$ é inconsistente

In [ ]:
```python
def k_inducao(inv, K, initial=None):
    w = 6
    for k in range(1, K + 1):

        with Solver(name="z3") as s:
            states = [declare(i) for i in range(k)]
            s.add_assertion(init(states[0], initial))
```

```
            for i in range(k - 1):
                s.add_assertion(trans(states[i], states[i + 1]))

            s.add_assertion(Or([Not(inv(st)) for st in states]))

            if s.solve():
                print(f"Falso. Contra-exemplo no caso base, k = {k}")
                model = s.get_model()
                w = len(str(model.get_value(states[-1]
['x'])).constant_value()))
                for i, st in enumerate(states):
                    pc_val = model.get_value(st['pc']).constant_value()
                    x_val = model.get_value(st['x']).constant_value()
                    y_val = model.get_value(st['y']).constant_value()
                    z_val = model.get_value(st['z']).constant_value()
                    print(f"Estado {i:>3} : pc = {pc_val:>1}, x = {x_val:>
{w}}, y = {y_val:>{w}}, z = {z_val:>{w}}")
                return False

        with Solver(name="z3") as s:

            states = [declare(i) for i in range(k + 1)]

            for i in range(k):
                s.add_assertion(trans(states[i], states[i + 1]))
                s.add_assertion(inv(states[i]))

            s.add_assertion(Not(inv(states[-1])))

            if not s.solve():
                print(f"Provado com k = {k}")
                return True

    print(f"Nenhum k até {K} prova a propriedade")
    return None


def inv(state):
    return Equals(BVAdd(BVMul(state['x'], state['y']), state['z']),
BVMul(state['a'], state['b']))
```

Teste

In [ ]:
```
k_inducao(inv,10)
```

```
Provado com k = 2
True
```

Parte c

k-indução com a condição $a < 2^{n/2} \wedge b < 2^{n/2}$ da propriedade NotEquals(state['pc'], Int(5))

Teste para mostrar a inseguraça do programa antes da restrição.

In [ ]:
```
def noterror(state):
    return NotEquals(state['pc'], Int(5))

k_inducao(noterror,10)
```

```
Falso. Contra-exemplo no caso base, k = 4
Estado   0 : pc = 0, x = 129, y =   8, z =   0
Estado   1 : pc = 1, x = 129, y =   8, z =   0
Estado   2 : pc = 2, x = 129, y =   8, z =   0
Estado   3 : pc = 5, x = 129, y =   8, z =   0
False
```

Função init com a condição $a < 2^{n/2} \wedge b < 2^{n/2}$

In [ ]:
```
def init2(state, i=None):
    constraints = [
        Equals(state['pc'], Int(0)),
        Equals(state['x'], state['a']),
        Equals(state['y'], state['b']),
        Equals(state['z'], BV(0, N))
    ]

    if SIGNAL:
        constraints.append(BVSGE(state['a'], BV(0,N)))
        constraints.append(BVSGE(state['b'], BV(0,N)))
        constraints.append(BVSLT(state['a'], BV(2**(N/2),N)))
        constraints.append(BVSLT(state['b'], BV(2**(N/2),N)))

    else:
        constraints.append(BVULT(state['a'], BV(2**int(N/2),N)))
        constraints.append(BVULT(state['b'], BV(2**int(N/2),N)))

    if i is not None:
        constraints.append(Equals(state['a'], i['a']))
        constraints.append(Equals(state['b'], i['b']))

    return And(constraints)
```

In [ ]:
```
def k_inducao2(inv, K, initial=None):
    w = 6
    for k in range(1, K + 1):

        with Solver(name="z3") as s:
            states = [declare(i) for i in range(k)]
            s.add_assertion(init2(states[0], initial))

            for i in range(k - 1):
```

```
                s.add_assertion(trans(states[i], states[i + 1]))

            s.add_assertion(Or([Not(inv(st)) for st in states]))

            if s.solve():
                print(f"Falso. Contra-exemplo no caso base, k = {k}")
                return False

        with Solver(name="z3") as s:

            states = [declare(i) for i in range(k + 1)]

            for i in range(k):
                s.add_assertion(trans(states[i], states[i + 1]))
                s.add_assertion(inv(states[i]))

            s.add_assertion(Not(inv(states[-1])))

            if not s.solve():
                print(f"Provado com k = {k}")
                return True

    print(f"Nenhum k até {K} prova a propriedade")
    return None
```

Teste com a condição

In [ ]:
```
k_inducao2(noterror, 100)
```

```
Provado com k = 47
True
```

Parte d

Quer-se provar que o algoritmo termina e termina corretamente usando verificação em traços infinitos; nomeadamente usando indução. Formalmente queremos provar que existe um estado onde se verifica $(z = 0)$; isto é, temos de provar que se verifica uma propriedade de animação $\mathsf{F}\,(z = 0)$.

In [ ]:
```
def bmc_eventually(prop,bound, initial=None):
    for k in range(1,bound+1):
         with Solver(name="z3") as s:

            # completar
            states = [declare(i) for i in range(k)]

            constraints = []

            constraints.append(init(states[0], initial))
            for i in range(k-1):
                constraints.append(trans(states[i], states[i+1]))
```

```
            for st in states:
                constraints.append(Not(prop(st)))

            loop = []

            for j in range(k-1):
                loop.append(trans(states[-1],states[j]))

            constraints.append(Or(loop))

            s.add_assertion(And(constraints))

            if s.solve():
                model = s.get_model()
                # print(model)
                for i, st in enumerate(states):
                    pc_val = model.get_value(st['pc'])
                    x_val = model.get_value(st['x'])
                    print(f"Estado {i} : pc = {pc_val}, x = {x_val}")

                for j in range(k):
                    if model.get_py_value(And(trans(states[-1],
states[j]))):
                        print(f"Loop de {k-1} para {j}")
                        break

                return False
    print(f"Nenhum contra-exemplo encontrado até {bound}")
    return
```

Teste

In [ ]:
```
def prop(state):
    return Or(stop(state), error(state))

bmc_eventually(prop, 50)
```

```
Nenhum contra-exemplo encontrado até 50
```

Uso de LLM

Neste trabalho, usamos o Claude como apoio neste trabalho. Em concreto:

- Explicação de funções relacinais para BitVecs, com e sem sinal;
- Debug de código: BV(1,N) trocado com Int(1), índices e chaves erradas em listas e dicionários, k-indução(parte b), função gera_traco, etc;
- Explicação do push e pop do solver e modificação função gera_traco com push e pop.